ЛАБОРАТОРНАЯ РАБОТА № 1.
АЛГОРИТМЫ СОРТИРОВКИ.

1. Постановка задачи

Отсортировать неупорядоченное множество чисел $A = \{a_1, \dots, a_n\}$ по возрастанию, то есть сформировать множество $\tilde{A} = \{a_{i1}, \dots, a_{in}\}$: $a_{i(j-1)} \leq a_{ij}$, $i, j = \overline{1, n}$, применив алгоритмы:

1) сортировки выбором;
2) быстрой сортировки;
3) по выбору (дополнительное задание).

2. Описание алгоритмов

Преобразуем исходное множество A в массив $A = \{A[0], \dots, A[N-1]\}$.

2.1. Сортировка выбором

Идея

На каждой итерации алгоритм находит наименьший элемент в списке и меняет его местами с первым неотсортированным элементом, затем переходит к следующей неотсортированной позиции и повторяет процесс до тех пор, пока весь список не будет отсортирован.

Исходный массив A разбивается на две части: итог A_1 и остаток A_2. Итог $A_1 = \{A_i[0], \dots, A_i[j-1]\}$ размещается в начале и должен быть упорядоченным, а остаток $A_2 = \{A_i[j], \dots, A_i[N]\}$ располагается сразу после итога и содержит неотсортированные числа исходного массива A.

Алгоритм

Шаг 1. Присваивается $j := 0$, полагая, что итоговый участок пуст.

Шаг 2. В остатке массива ищется минимальный и меняется местами с первым элементом остатка (j-м элементом массива). После этого значение увеличивается на единицу j++, расширяя отсортированную часть (итог).

Шаг 3. Если $j < N - 1$, повторяется шаг 2, в противном случае алгоритм завершается, так как итог становится равным всему массиву.

Резюме

Алгоритм сортировки выбором имеет временную сложность $O(n^2)$. В большинстве случаев он эффективнее, чем алгоритм пузырьковой сортировки, но его производительность снижается при работе с большими наборами данных. Преимуществом является простота реализации.

2.2. Быстрая сортировка

Идея

Алгоритм быстрой сортировки, основанный на подходе «разделяй и властвуй», является одним из самых эффективных алгоритмов сортировки. Выбирается опорный элемент массива (удобнее взять медиану), после чего массив разбивается на два подмножества: одно с элементами, меньшими опорного, другое с элементами, превышающими опорный. Затем рекурсивно применяется эта же процедура к двум подмножествам, пока весь массив не будет отсортирован.

Обозначим первый и последний индексы элементов массива A соответственно *First* и *Last*. Введем еще два индекса: *Index*, указывающий на переставляемый элемент, и *ContrIndex*, указывающий на элемент, который сравнивается с переставляемым. В процессе вычислений вводится переменная h – шаг движения индексов навстречу друг другу, используемая для обозначения направления движения индекса *ContrIndex* ($h = 1 \vee h = -1$), и логическая переменная *Condition* (равная TRUE либо FALSE), используемая для изменения условия сравнения на противоположное при обратном движении индекса *ContrIndex*.

Алгоритм

Шаг 1. Если $First \geq Last \Longrightarrow$ выход из алгоритма. В противном случае полагаются $h := 1,\ Condition := TRUE,\ Index := First,\ ContrIndex := Last$ и выполняются шаги 2–3.

Шаг 2. Пока $Index \neq ContrIndex$, выполняются шаги 2a–2b.

Шаг 2a. Если справедливо $((A[Index] > A[ContrIndex]) = Condition)$, то переставляются:

1) элементы, на которые указывают *Index* и *ContrIndex* ($Val := A[Index]$, $A[Index] := A[ContrIndex],\ A[ContrIndex] := Val$)

2) вспомогательные индексы массивов ($Val := Index, Index := ContrIndex, ContrIndex := Val$).

Затем меняется направление движения ($h := -h$) и условие сравнения ($Condition := not\ Condition$). В процессе таких перестановок слева от переставляемого элемента всегда будут находиться меньшие значения, а справа – б***о***льшие.

Шаг 2b. Сдвигается вспомогательный индекс массива *ContrIndex* навстречу индексу *Index*, то есть $ContrIndex := ContrIndex + h$.

Шаг 3. Перед выполнением этого шага элемент *A*[*Index*] находится на нужном месте, а *Index=ContrIndex*. Таким образом, исходный массив разбит на три части, в первой из которых значения меньше величины *A*[*Index*], во второй – больше *A*[*Index*] и сам элемент *A*[*Index*]. Для дальнейшего упорядочивания массива осуществляется рекурсивное обращение к алгоритму быстрой сортировки два раза: для первой и второй частей массива. Каждый раз длина сортируемых участков будет уменьшаться, в итоге весь массив будет полностью отсортирован.

Резюме

Алгоритм быстрой сортировки имеет среднюю временную сложность $O(n \log n)$, что делает его эффективным для больших наборов данных. В худшем случае его производительность может снизиться до $O(n^2)$, если точка опоры выбрана неблагоприятно.

3. Задание

Осуществить программную реализацию приведенных выше алгоритмов и произвольного алгоритма (по выбору студента) и протестировать программу:

а) на произвольном сгенерированном наборе данных, состоящем из n элементов (n вводится с клавиатуры). В массиве должны быть повторяющиеся элементы. Вывести результаты сортировки, время работы алгоритма и количество перестановок.

б) на массиве, содержащем сведения о возрасте людей, совершивших преступления в Москве в 2021 году (файл *Москва_2021.txt*). Вывести время работы алгоритма и количество перестановок. После этого удалить дубликаты и вывести результаты сортировки.